Page No. :
Date :

```
public List<Integer> eventualSafe(int V,
                List<List<Integer>> adj) {
int vis[] = new int[V];
int pathVis[] = new int[V];
int check[] = new int[V];

for(int i=0; i<V; i++){
    if(vis[i] == 0){
        dfsCheck(i, adj, vis, pathVis, check);
    }
}

List<Integer> safeNodes = new ArrayList<>();

for(int i= 0; i<V; i++){
    if(check[i] == 1){
        safeNodes.add(i);
    }
}

return safeNodes;
}
```

TC- ~~O(3N)~~ O(V+E)

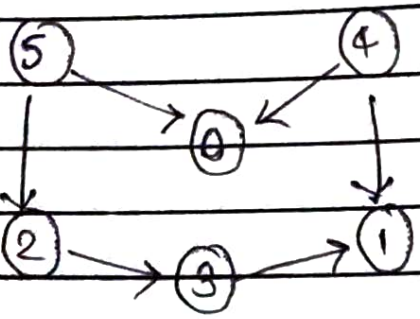
SC- O(3N)

Page No. :

Date :

15. Topological Sort Algorithm – (DFS)

DAG (Directed Acyclic Graph)



5 4 2 3 1 0

Ans. ∞

4 5 2 3 1 0

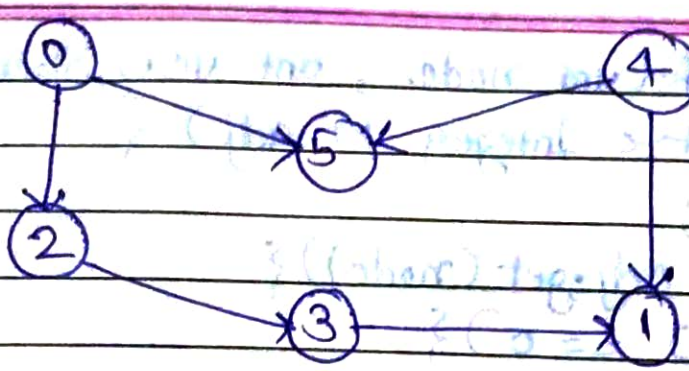
Linear ordering of vertices such that if there is an edge between u & v, u appears before v in that ordering.

5 → 0
4 → 0
5 → 2
2 → 3
3 → 1
4 → 1

Dry Run :-

① vis[] = | 0 | 0 | 0 | 0 | 0 |
0 1 2 3 4

0 → {}
1 → {}
2 → {3}
3 → {1}
4 → {0,1}
5 → {0,2}



Adjacent Node —

0 → {2, 5}
1 → { }
2 → {3}
3 → {1}
4 → {1, 5}
5 → { }

① (0) → (2)

1	1	1	1			vis
0	1	2	3	4	5	

2
3
1

② 0 → (5)

1	1	1	1			vis
0	1	2	3	4	5	

5
2
3
1

C

Page No. :
Date :

Code:-

```
private static void dfs(int node, int vis[], Stack<Integer> st,
    ArrayList<ArrayList<Integer>> adj) {
    vis[node] = 1;
    for (int it: adj.get(node)){
        if (vis[it] == 0){
            dfs(it, vis, st, adj);
        }
    }
    st.push(node);
}

public static int topoSort() (int V, ArrayList<ArrayList
                                <Integer>> adj) {
    int vis[] = new int[V];
    Stack<Integer> st = new Stack<Integer>();

    for(int i = 0; i<V; i++){
        if (vis[i] == 0) {
            dfs(i, vis, st, adj);
        }
    }
    int ans[] = new int[V];
    int i = 0;
    while (!st.isEmpty()) {
        ans[i++] = st.peek();
        st.pop();
    }
    return ans;
}
```

TC - O(V+E)
SC - O(2N)

Page No. :
Date :

③ 1, 2, 3, already visited

1	1	1	1		1	visited
0	1	2	3	4	5	

5
2
3
1

④ ④ → {1, 5}

दोनों already visited थे।

0
4
5
2
3
1

Stack

⇒ 0, 4, 5, 2, 3, 1 Ans.

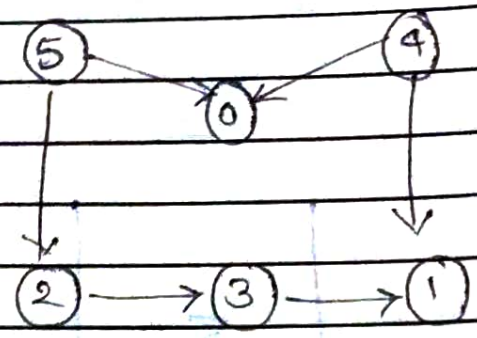
Co

publ

Page No. :

Date :

16 Topological Sorting (Kahn's Algorithm | BFS)



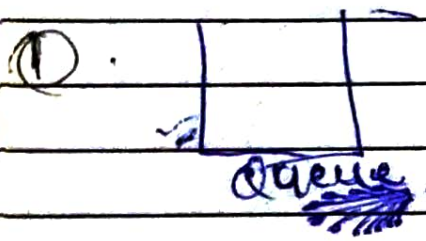
5 4 0 2 3 1

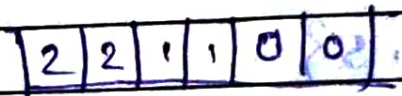
or

4 5 2 3 1 0

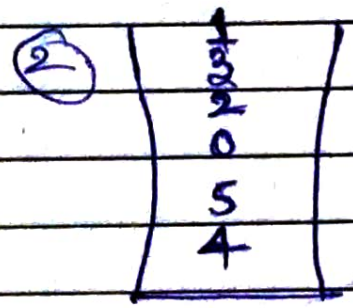
* DAG (Directed Acyclic Graph)

Dry Run:-





Indegree - No. of incoming edges to a node



(i) Insert All nodes with indegree 0

0 →

1 →

2 → {3}

3 → {1}

4 → {0, 1}

5 → {0, 2}

Page No.
Date :

```
public static int[] topoSort(int V, ArrayList<ArrayList
        <Integer>> adj) {
    int indegree[] = new int[V];
    for(int i = 0; i < V; i++) {
        for(int it: adj.get(i)) {
            indegree[it]++;
        }
    }

    Queue<Integer> q = new LinkedList<Integer>();
    for(int i = 0; i < V; i++) {
        if(indegree[i] == 0) {
            q.add(i);
        }
    }
```

SC - O(2N)
TC - O(V+E)

```
    int topo[] = new int[V];
    int i = 0;
    while(!q.isEmpty()) {
        int node = q.peek();
        q.remove();
        topo[i++] = node;

        for(int it: adj.get(node)) {
            indegree[it]--;
            if(indegree[it] == 0) {
                q.add(it);
            }
        }
    }
    return topo;
}
```

pub

Date :

① Queue: 5, 4 (5 on top)

Take out ④

0 →
1 →
2 → {3}
3 → {1}
4 → {0,1}
5 → {0,2}

0	1	2	3	4	5
~~1~~	~~2~~	1	1	0	0
	1				

→ indegree

② Queue: 0, 5 (0 on top)

Take out ⑤

0	1	2	3	4	5
~~1~~	1	~~1~~	1	0	0
0		0			

→ indegree

③ Queue: 2, 0 (2 on top)

Take out ⑥

0	1	2	3	4	5
0	1	0	1	0	0

→ indegree

④ Queue: 3, 2 (3 on top)

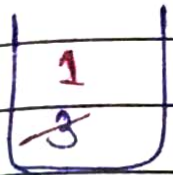
Take out ②

0	1	2	3	4	5
0	1	0	~~1~~	0	0
			0		

→ indegree

Page No. :
Date :

(5)



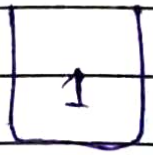
Queue

Take out (3)

0	1	2	3	4	5
0	+	0	0	0	0
	0				

→ indegree

(6)



Take out (1)

0	1	2	3	4	5
0	0	0	0	0	0

→ indegree

450231 ⇒ Ans.

Logic –

01 – Insert all nodes with indegree 0 into the Queue.

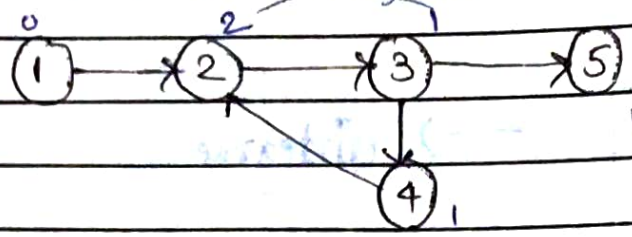
Take them out of the queue

& reduce the indegree of the adjacent Node

If the adjacent Node indegree has become zero so now you can go into the Queue.

Page No. :
Date :

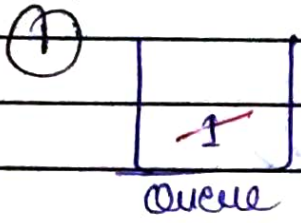
17. Detect a cycle in Directed Graph –
{Topological Sort | Kahn's Algo | BFS}



Indegrees

Start with the Node of Indegree 0

Dry Run :–



Take out (1)

1 → {2}
2 → {3}
3 → {4, 5}
4 → {2}
5 → {}

1	2	3	4	5	
0	~~2~~ 1	1	1	1	→ Indegrees

{if TopoSort < N} ⇒ It means contains cycle

TopoSort works on DAG (Directed Acyclic Graph)
If it doesn't work it means

If you are able to generate toposort it means it does not have a cycle.

If you are not able to generate topo sort it means it has a cycle.

Page No. :
Date :

Code:-

```
public boolean isCyclic(int V, ArrayList<ArrayList<Integer>> adj){
    int indegree[] = new int[V];
    for(int i=0; i<V; i++){
        for(int it : adj.get(i)){
            indegree[it]++;
        }
    }

    Queue<Integer> q = new LinkedList<Integer>();
    for(int i=0; i<V; i++){
        if(indegree[i] == 0){
            q.add(i);
        }
    }

    int cnt = 0;
    while(!q.isEmpty()){
        int node = q.peek();
        q.remove();
        cnt++;

        for(int it : adj.get(node)){
            indegree[it]--;
            if(indegree[it] == 0){
                q.add(it);
            }
        }
    }
    if(cnt == V) return false;
    return true;
}
```

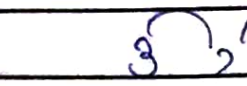
TC- O(V+E)
SC- O(2N)

Page No. :
Date :

18. Course Schedule I and II -/ Pre-requisite Tasks -

Ex-01

1. 0
2. 1
3. 2

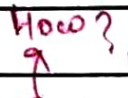


⇒ ans ⇒ yes.

first you need to perform task 1 then task 0
task 2 then task 1
task 3 then task 2.

Ex-02

1. 2
4. 3
2. 4
4. 1



cyclic dependency

ans ⇒ No

(01) Create adjacency list

(02) Find Indegree

(03) Store Indegree in the Queue.

(04) Now traverse Queue & Store whose indegree becomes zero

(05) Check the condition at last

Page No. :
Date :

ode :- I

```
public boolean isPossible (int V, int prerequisites[][]){
  ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
  for (int i = 0; i<V; i++ ){
     adj.add (new ArrayList<>());
  }
  int n = ~~prerequisites.length~~ V;
  for (int i = 0; i<n; i++) {
     adj.get (prerequisites[i][0]).add (prerequisites[i][1]);
  }

  int indegree[] = new int[V];
  for (int i = 0; i<V; i++) {
     for (int it : adj.get(i)) {
        indegree[it] ++;
     }
  }

  Queue<Integer> q = new LinkedList<Integer>();
  for (int i =0; i<V; i++) {
     if (indegree[i] == 0) {
        q.add (i);
     }
  }

  List<Integer> topo = new ArrayList<Integer> ();
  while (!q.isEmpty()) {
     int node = q.peek();
     q.remove();
     topo.add (node);
```

pub

Date :

```
        for(int it : adj.get(node)) {
            indegree[it]--;
            if(indegree[it] == 0) {
                q.add(it);
            }
        }
    }

    if(topo.size() == V){
        return true;
    }

    return false;
}
```

TC- O(V+E)

SC- O(2N)

Code II :-

Page No. :
Date :

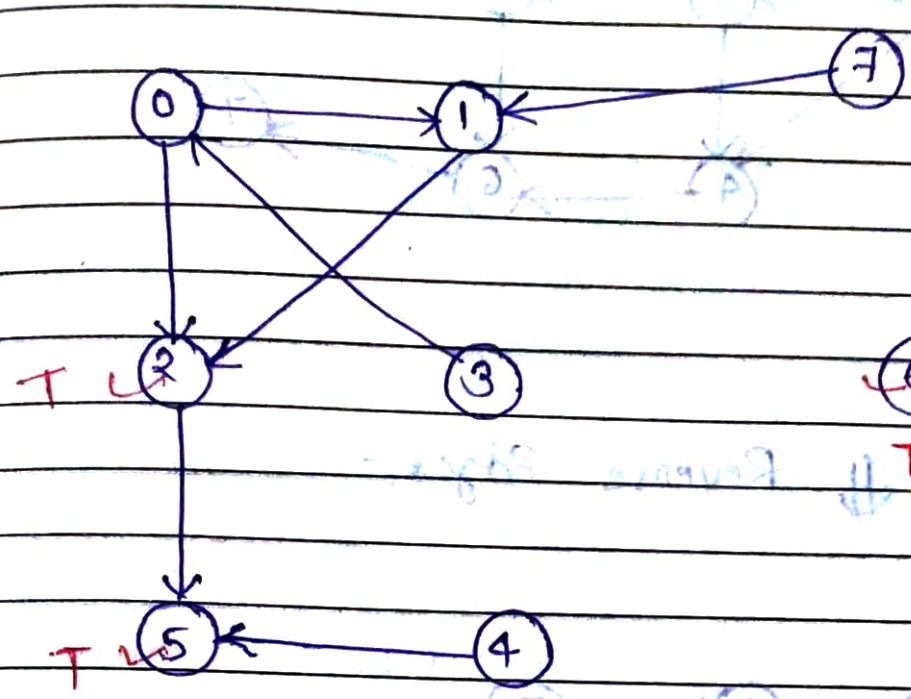
```
public static int findOrder(int n, int m,
        ArrayList<ArrayList<Integer>> prerequisites) {
ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
for(int i = 0; i<n; i++) {
    adj.add(new ArrayList<>());
}

for (int i=0; i<m; i++) {
    adj.get(prerequisites.get(i).get(1)).add
            (prerequisites.get(i).get(0));
}

int indegree[] = new int[n];
for(int i = 0; i<n; i++) {
    for (int it: adj.get(i)) {
        indegree[it]++;
    }
}

Queue<Integer> q = new LinkedList<Integer>();
for(int i = 0; i<n; i++) {
    if (indegree[i] == 0) {
        q.add(i);
    }
}

int topo[] = new int[n];
int ind = 0;
```

```
while (!q.isEmpty()) {
    int node = q.peek();

    q.remove();
    topo[ind++] = node;

    for (int it : adj.get(node)) {
        indegree[it]--;
        if (indegree[it] == 0) {
            q.add(it);
        }
    }
}

if (ind == n) {
    return topo;
}
int arr[] = {};
return arr;
}
```

TC- O(V+E)
SC ~ O(2N)

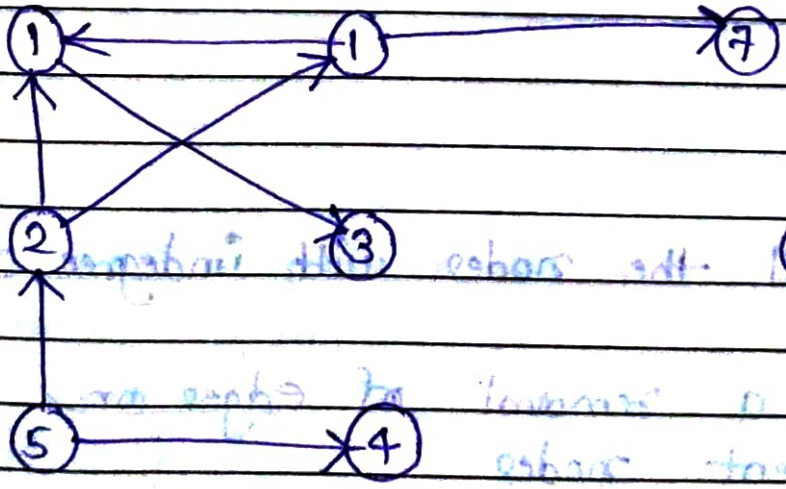
Page No. :
Date :

19. Find Eventual Safe States - BFS



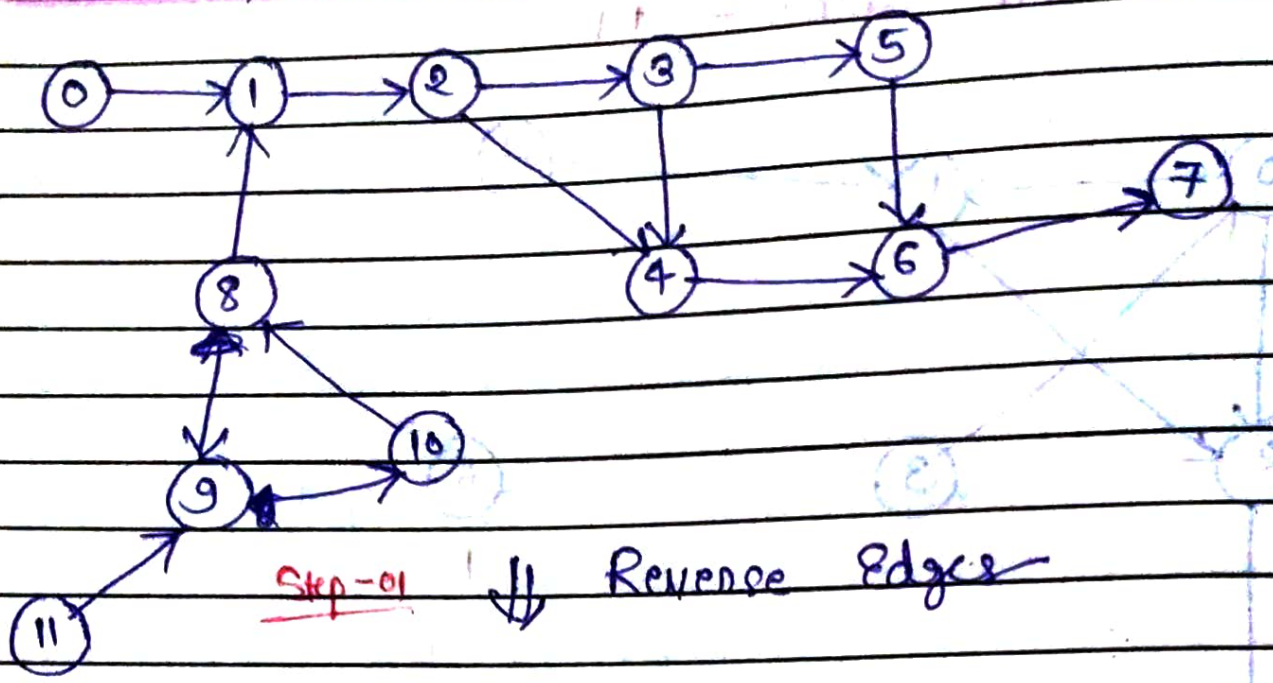
Terminal Node - Node that has no outgoing edges

01st Step - Reverse all Edges

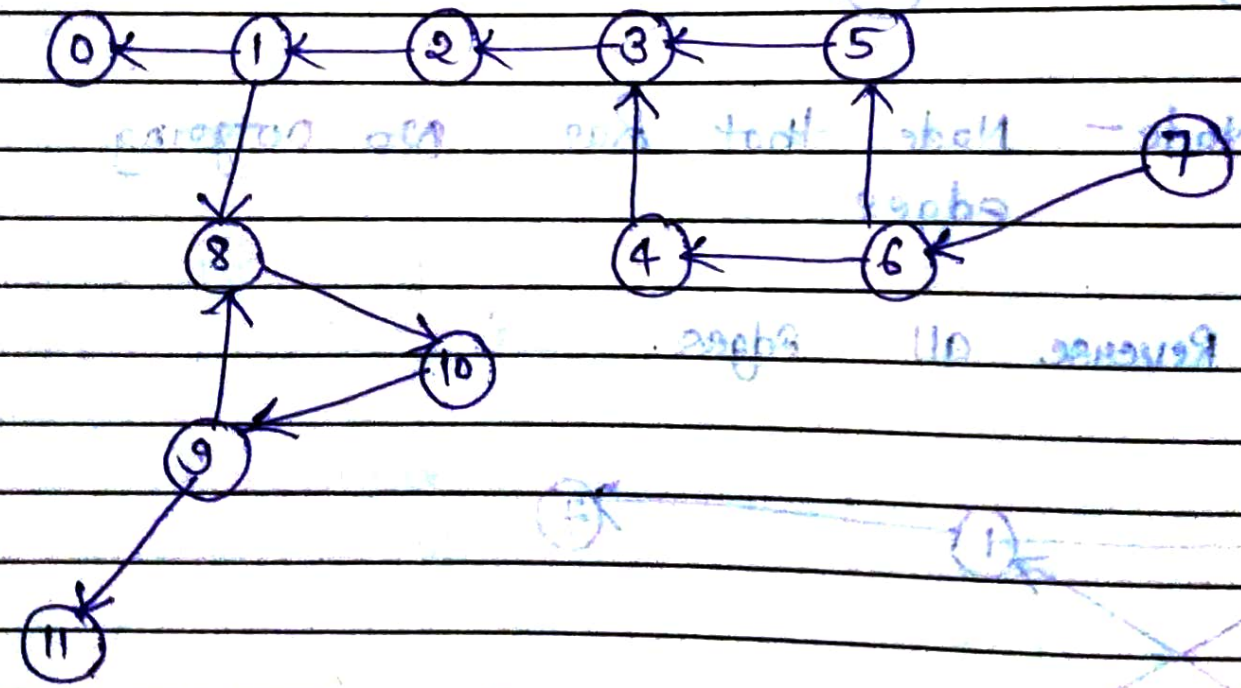


Page No. .....
Date :

Let's Dry Run On This Example —



Step-01 ⇓ Reverse Edges



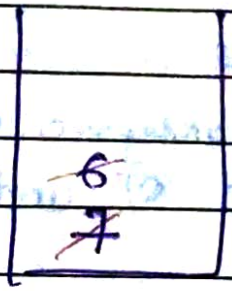
Step 02 - Get all the nodes with indegree 0.

Step 03 = Do a removal of edges on adjacent nodes.

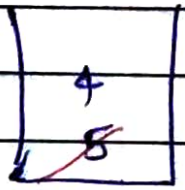
Page No. :
Date :

① node = 7

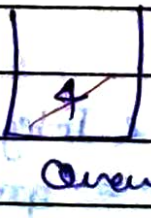
② node = 6 (Remove the indegree of adjacent node)
which is {5, 4}
Now 5 & 4 की indegree 0 हो गयी है,
put them into queue.



③ node = 5
Reduce indegree of {3}

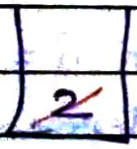


④ node = 4
Reduce indegree of {3, 2}
Now 3 की indegree 0, put it into queue



⑤ node = 3
Reduce indegree of {2}
Now 2 की indegree 0, put it into queue

⑥ node = 2
Reduce indegree of {0}
Now 0 की indegree 0, put it into queue



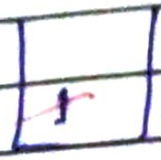
pul

Date :

(7) node = 1

Reduce undegree of {0,8}

Now, 0 of indegree 0



(8) node = 0

0 does have any adjacent right now, means 0 ka also done

}

pr

(9)

```
public List<Integer> eventualSafeNodes(int[][] graph) {
    int V = graph.length;

    List<List<Integer>> adjRev = new ArrayList<>(V);
    for (int i = 0; i < V; i++) {
        adjRev.add(new ArrayList<>());
    }

    // Outdegree array (acts like indegree but on reversed graph)
    int outdegree[] = new int[V];

    for (int i = 0; i < V; i++) {
        outdegree[i] = graph[i].length;
        for (int neighbor : graph[i]) {
            adjRev.get(neighbor).add(i);
        }
    }
}
```

//Code:-

Page No. :
Date :

```
Queue<Integer> queue = new LinkedList<>();
List<Integer> safeNodes = new ArrayList<>();

for (int i = 0; i < V; i++) {
    if (outdegree[i] == 0) {
        queue.offer(i);
    }
}

// Kahn's algorithm on reversed graph
while (!queue.isEmpty()) {
    int node = queue.poll();
    safeNodes.add(node);

    for (int parent : adjRev.get(node)) {
        outdegree[parent]--;
        if (outdegree[parent] == 0) {
            queue.offer(parent);
        }
    }
}

Collections.sort(safeNodes);
return safeNodes;
}
```

TC-

SC-

20. Alien dictionary — N = 4 —

b a a
a b c d
a b c a
c a b
c a d

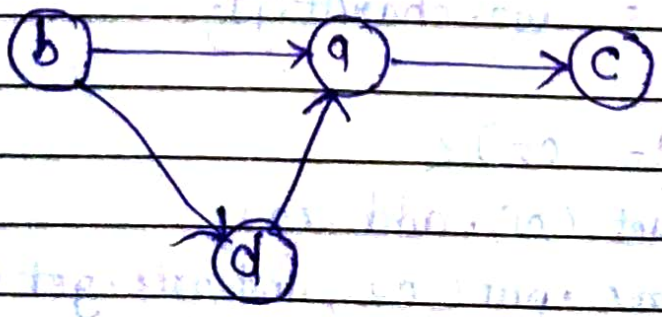
find out the alien order ?

Ans. bdac

— Solving —

01 a b c d
0 1 2 3

{ Try to represent all aphabets in terms of no. }



Now find Topological Sort

e { if e or more alphabet would have presented }

bdace

Ans.

When the order is not possible — Dictionary is wrong

① abcd, abc ⇒ Not possible

② Cyclic Dependency

Page No. :-

Date :

// Code :-

```
public String alienOrder(String words[]) {
    Map<Character, List<Character>> adj = new HashMap<>();
    Map<Character, Integer> indegree = new HashMap<>();

    for (String word : words) {
        for (char c : word.toCharArray()) {
            adj.putIfAbsent(c, new ArrayList<>());
            indegree.putIfAbsent(c, 0);
        }
    }

    for (int i = 0; i < words.length - 1; i++) {
        String w1 = words[i];
        String w2 = words[i+1];
        int len = Math.min(w1.length(), w2.length());
        boolean foundDiff = false;

        for (int j = 0; j < len; j++) {
            char c1 = w1.charAt(j);
            char c2 = w2.charAt(j);

            if (c1 != c2) {
                adj.get(c1).add(c2);
                indegree.put(c2, indegree.get(c2) + 1);
                foundDiff = true;
                break;
            }
        }

        if (!foundDiff && w1.length() > w2.length()) {
            return "";
        }
    }
```

Page No. :
Date :

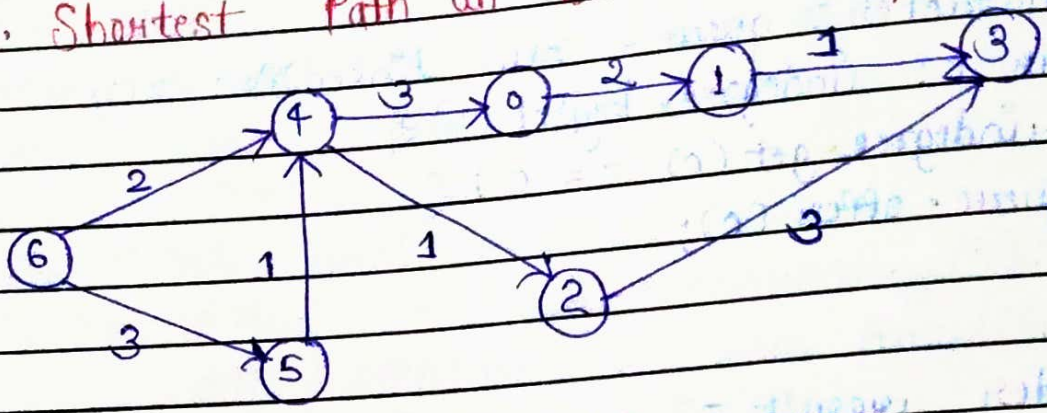
```
Queue<Character> queue = new LinkedList<>();
for (char c: indegree.keySet()){
    if (indegree.get(c) == 0){
        queue.offer(c);
    }
}
StringBuilder result = new StringBuilder();

while (!queue.isEmpty()){
    char curr = queue.poll();
    result.append(curr);

    for (char next: adj.get(curr)){
        indegree.put(next, indegree.get(next)-1);
        if (indegree.get(next) == 0){
            queue.offer(next);
        }
    }
}
if (result.length() != indegree.size()){
    return "";
}
return result.toString();
}
```

TC -

SC -

Page No. :
Date :

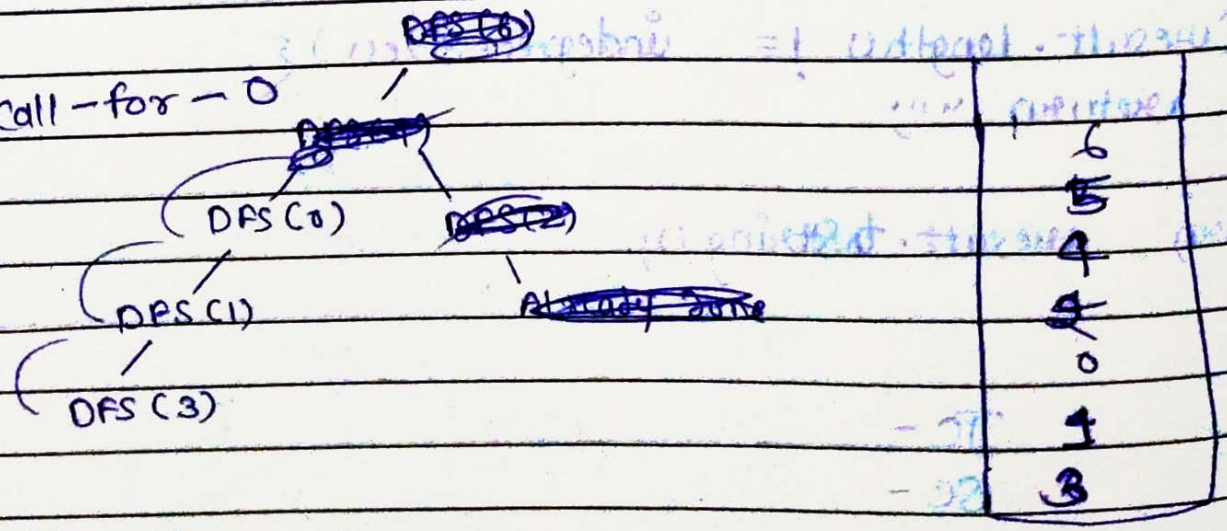
21. Shortest Path in Directed Acyclic Graph –



Adjacency list –

0 → {1, 2}
1 → {3, 1}
2 → {3, 3}
3 →
4 → {0, 3} {2, 1}
5 → {4, 1}
6 → {4, 2} {5, 3}

01 Step – 01 – Do a Topo Sort on the Graph



Call for – 2

Page No. :
Date :

Step-02 Take the nodes out of the Stack & Take Distance arr & Start relaxation.

dist[] = | -∞ | -∞ | -∞ | -∞ | -∞ | -∞ | 0 |
0 1 2 3 4 5 6

Note :- source node का dist == 0 करना

① node = 6 , dist = 0

+3 → node = 5

+2 → node = 4

5
4
2
0
1
3

dist[] = | | | | | 2 | 3 | 0 |
0 1 2 3 4 5 6

② node = 5 , dist = 3

+1 → node = 4 ⇒ 4

5
4
2
0
1
3

dist[] = | | | | | 2 | 3 | 0 |
0 1 2 3 4 5 6

{ for 4 I already have better option which is 2 dist < 4 dist }

Page No. :

Date :

③ node = 4, 방향 = 2

+3 / +1

node = 0 / node = 2

⇒ 5 ⇒ 3

4
2
0
1
3

Stack

방향 = | 5 | | 3 | | 2 | 3 | 0 |

0 1 2 3 4 5 6

④ node = 2, 방향 = 3

+3

node = 3

⇒ 6

2
0
1
3

Stack

방향 = | 5 | | 3 | 6 | 2 | 3 | 0 |

0 1 2 3 4 5 6

⑤ node = 0, 방향 = 5

+2

node = 1

⇒ 7

0
1
3

방향2 | 5 | 7 | 3 | 6 | 2 | 3 | 0 |

0 1 2 3 4 5 6

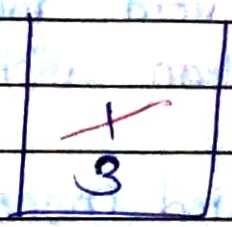
Page No. :
Date :

(1) node = 1, arr = 7

/ +1

node = 3

⇒ 8

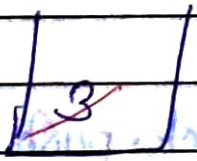


arr [] =

5	7	3	6	7	8	9
0	1	2	3	4	5	6

(2) node = 3

No adjacent nodes



Page No. :
Date :

// Code :-

```
private void topoSort ( int node, List< List<int[]>> adj,
    boolean visited[], Stack< Integer > Stack ) {

    visited [node] = true;

    for (int edge[] : adj.get(node)) {
        int next = edge[0];
        if ( !visited[next] ) {
            topoSort (next, adj, visited, stack);
        }
    }

    stack.push (node);
}

public int[] ShortestPath (int N, int M,
                           int edges[][] ) {

    list < list < int[] >> adj = new ArrayList <> ();
    for (int i = 0; i < N; i++) {
        adj.add (new ArrayList <> ());
    }

    for (int i = 0; i < M; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int wt = edges[i][2];
        adj.get(u).add (new int[] {v, wt});
    }
```

Page No. :
Date :

```
// topological Sort
boolean visited[] = new boolean[N];
Stack<Integer> Stack = new Stack<>();

for(int i=0; i<N; i++){
    if(!visited[i]){
        topoSort(i, adj, visited, Stack);
    }
}

// Distance Array
int dist[] = new int[N];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[0] = 0;

// Relax edges in topo order
while(!Stack.isEmpty()){
    int node = Stack.pop();

    if(dist[node] != Integer.MAX_VALUE){
        for(int edge[]: adj.get(node)){
            int v = edge[0];
            int wt = edge[1];
            if(dist[node] + wt < dist[v]){
                dist[v] = dist[node] + wt;
            }
        }
    }
}
```

Page No. :
Date :

```
// Replace unreachable nodes with -1
for(int i = 0; i < N; i++){
    if(dist[i] == Integer.MAX_VALUE){
        dist[i] = -1;
    }
}
return dist;
}
```

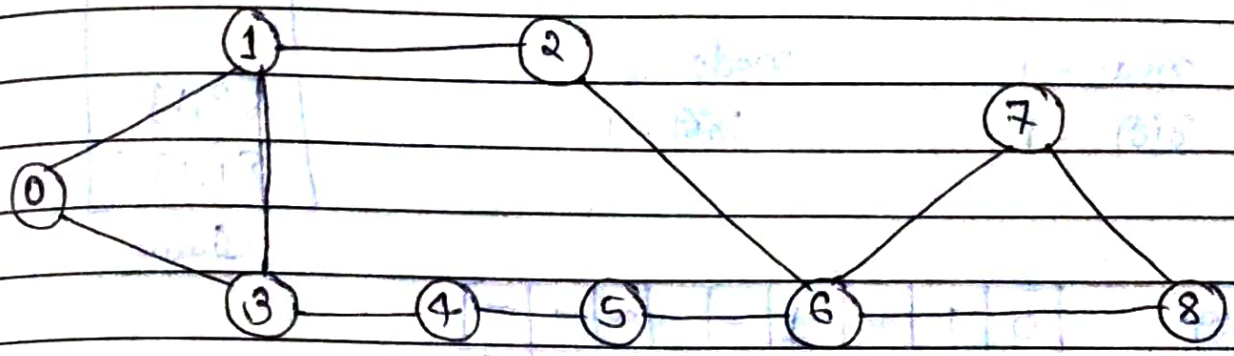
TC -

SC -

Page No. :
Date :

22. Shortest Path In Undirected Graph with Unit Weights – {BFS}

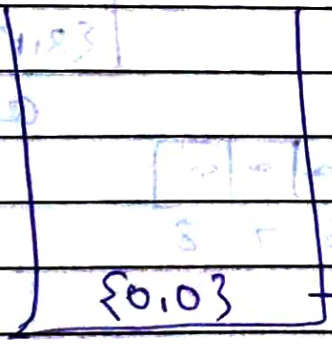
Edge weight = 1



src = 0

— Approach —

Initially :



→ src = {0,0}

Queue

0 → {1,3}
1 → {0,2,3}
2 → {1,6}
3 → {0,4}
4 → {3,5}
5 → {4,6}
6 → {2,5,7,8}
7 → {6,8}
8 → {6,7}

0

dist – | 0 | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |

0 1 2 3 4 5 6 7 8

जो भी src given है उसकी दूरी 0 करदो

Date :

① node = 0, dist = 0

node = 1, dist = 1 | node = 3, dist = 1

{3,1}
{1,1}

Queue

dist = | 0 | 1 | ∞ | 1 | ∞ | ∞ | ∞ | ∞ | ∞ |
0 1 2 3 4 5 6 7 8

② node = 1, dist = 1

node = 0, dist = 2 | node = 2, dist = 2 | node = 3, dist = 2

{2,2}
{3,1}

Queue

dist = | 0 | 1 | 2 | 1 | ∞ | ∞ | ∞ | ∞ | ∞ |
0 1 2 3 4 5 6 7 8

③ node = 3, dist = 1

node = 0, dist = 0 | node = 4, dist = 2

{4,2}
{2,2}

Queue

dist = | 0 | 1 | 2 | 1 | 2 | ∞ | ∞ | ∞ | ∞ |
0 1 2 3 4 5 6 7 8

Page No. :
Date :

④ node = 2, dist = 2

node = 1, dist = 3

node = 6, dist = 3

{6,3}
{4,2}

Queue

dist =

0	1	2	3	4	5	6	7	8
0	1	2	1	2	∞	3	∞	∞

⑤ node = 4, dist = 2

node = 3, dist =

node = 5, dist = 3

{5,3}
{6,3}

Queue

dist =

0	1	2	3	4	5	6	7	8
0	1	2	1	2	3	3	∞	∞

⑥ node = 6, dist = 3

node = 2, dist = 4

node = 5, dist = 4

node = 7, dist = 4

node = 8, dist = 4

{8,4}
{7,4}
{5,3}

Queue

dist =

0	1	2	3	4	5	6	7	8
0	1	2	1	2	3	3	4	4

Page No. .....
Date :

(7) node = 5, dist = 3

node = 4, dist = 4

node = 6, dist = 4

{8,4}
{7,4}

Queue

dist =

0	1	2	1	2	3	3	4	4
0	1	2	3	4	5	6	7	8

(8) node = 7, dist = 4

node = 6, dist = 5

node = 8, dist = 5

{8,4}

Queue

(9) node = 8, dist = 4

node = 6, dist = 5

node = 7, dist = 5

0

Queue

dist =

0	1	2	1	2	3	3	4	4
0	1	2	3	4	5	6	7	8

Now Queue Become Empty...

//Code:-

Page No. :
Date :

```
public int [] ShortestPath (int edges[][], int N,
                            int m, int src) {

List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i<N; i++){
    adj.add (new ArrayList<>());
}

for (int edge [] : edges){
    adj.get (edge[0]).add (edge[1]);
    adj.get (edge[1]).add (edge[0]);
}

int dist [] = new int [N];
Arrays.fill (dist, (int) 1e9);

dist [src] = 0;

Queue<Integer> queue = new LinkedList<>();
queue.add (src);

while (!queue.isEmpty ()){
    int node = queue.poll();

    for (int neighbor: adj.get (node)){
        if (dist [node] + 1 < dist [neighbor]){
            dist [neighbor] = dist [node] + 1;
            queue.add (neighbor);
        }
    }
}
```

Date :

```
for (int i = 0; i < N; i++) {
    if (dist[i] == (int) 1e9) {
        dist[i] = -1;
    }
}

return dist;
```

TC -

SC -

Page No. :

Date :

23. Word Ladder - I

Input - wordlist = {"des", "der", "dfr", "dgt", "dfs"}

startWord = "der", targetWord = "dfs"

Output - 3

Explanation - "der" → "dfr" → "dfs"

— DRY RUN —

Initially :- wordlist = [hot, dot, dog, lot, log, log]

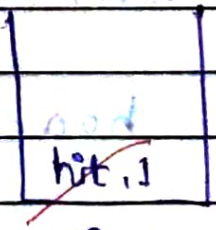
startWord = hit endWord = log

(1) hit, 1

(2) first you take 'h' & check

⇒ ait, bit, cit, dit ... zit

None of them exist in wordlist



Queue

(2) Now you take 'o' & check

⇒ hat, hbt, hct ... hot ... hzt

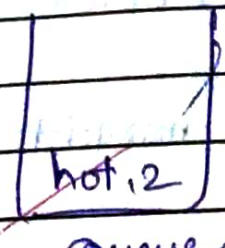
hot exist in wordlist

Page No. :
Date :

(2) hit, 1

/

hot, 2



queue

Now take 't' & check

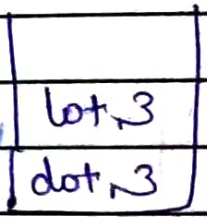
hia, hib, hic .... hiz

None of them exist in the wordlist

(3) hot, 2

/ \

dot, 3 lot, 3



queue

'h' → aot, bot, cot, dot ... lot ... zot ⇒ wordlist में है

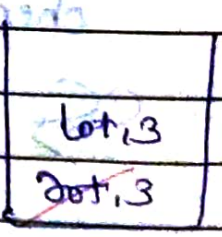
'o' → hat, hbt, hct ... hzt ⇒ None ✗

't' → hoa, hob, hoc ... hoz ⇒ None ✗

(4) dot, 3

/

dog, 4



'd' → aot, bot, cot ... zot → None ✗

'o' → dat, dbt ... dzt → None ✗

't' → doa, dob, doc, dog ... doz →

exist

Page No. :
Date :

(5) lot, 3
log, 4

dog, 4
~~lot, 3~~

Queue.

'l' → aot, bot ... zot → None ✗
'o' → lat, lbt, lct ... lzt → None ✗
't' → loa, lob, loc ... log .. loz →
possible

(6) dog, 4
log, 5

log, 4
~~dog, 4~~

'd' → aog, bog, cog .... zog exist
'o' → dag, dbg, dcg .... dzg
'g' → dog, dob, doc ... dog ... doz

(7) log, 4
cog → already in ~~set~~.

cog, ~~5~~
~~log, 4~~

(8) cog, 5 ⟹ Ans ⇒ 5

// Code :—

Page No. :
Date :

```
public int wordLadderLength(String startWord,
    String targetWord, List<String> wordList) {

Set<String> set = new HashSet<>(wordList);

if (!set.contains(targetWord)) {
    return 0;
}

Queue<String> queue = new LinkedList<>();
queue.add(startWord);
set.remove(startWord);

int steps = 1;

while (!queue.isEmpty()) {
    int size = queue.size();

    for (int k = 0; k < size; k++) {
        String word = queue.poll();

        if (word.equals(targetWord)) {
            return steps;
        }

        char arr[] = word.toCharArray();
```

Page No. :
Date :

```
for (int i = 0; i < arr.length; i++) {
    char original = arr[i];

    for (char ch = 'a'; ch <= 'z'; ch++) {
        arr[i] = ch;
        String newWord = new String(arr);

        if (set.contains(newWord)) {
            queue.add(newWord);
            set.remove(newWord);
        }
    }

    arr[i] = original;
}

steps++;
}
return 0;
}
```

TC - $O(N * L * 26) \approx O(N * L)$

- N : no. of words in wordList
- L : length of each word

SC - $O(2N)$

- set, queue

Page No. :
Date :

24. Word Ladder - II

Input :- wordList - [pat, bat, pot, poz, coz]

startWord - bat endWord - coz

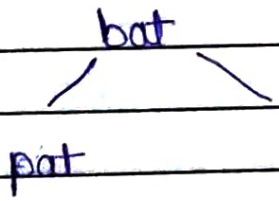
Ans ⇒ bat → pat → pot → poz → coz
bat → bot → pot → poz → coz

Intuition

(1)

wordList - [pat, bat, pot, poz, coz]

startWord - bat endWord - coz



~~{bat, pot, pot}~~
{bat, bot}
{bat, pat}
~~{bat}~~

Queue

Page No. :

Date :

② wordList = [pot , poz , coz]

startWord = bat endWord = coz

{ bat , pat }

(i) pat ← pot

{bat, pat, pot}
~~{bat, pat}~~, bot}
~~{bat, pat}~~

Queue

(ii) { bat , bot }

bot ← pot

{bat, bot, pot}
{bat, pat, pot}

Queue

③ wordList - [poz, coz]

startWord = bat endWord = coz

{ bat , pat , pot }

(i) pot ← poz

{bat, pat, pot, poz}
{bat, bot, pot}
~~{bat, pat, pot}~~

Queue

Page No. :
Date :

(ii) {bat, bot, pot}

pot
↙
poz

{bat, bot, pot, poz}
{bat, pot, pot, poz}
Queue.

(4) wordList = [coz]
startWord = bat endWord = coz

(i) {bat, pat, pot, poz}

poz
↙
coz

{bat, bot, pot, poz}
{bat, pat, pot, poz}
Queue

(ii) {bat, pat, pot, poz}

poz
↓
coz

(5) bat, pat, pot, poz, coz

⇓
ans

{bat, bot, pot, poz, coz}
{bat, pat, pot, poz, coz}
Queue.

// Code –

Page No. :

Date :

```
public List<List<String>> findSequences(String beginWord,
        String endWord, List<String> wordList){

    Set<String> set = new HashSet<>(wordList);
    // Queue. Stores paths
    Queue<List<String>> queue =
                    new LinkedList<>();
    queue.offer(new ArrayList<>(List.of(beginWord)));

    // Words used at current BFS level
    List<String> usedAtCurrentLevel = new ArrayList<>();
    usedAtCurrentLevel.add(beginWord);

    int currentLevel = 1;

    // Result Storing all Shortest sequences
    List<List<String>> result = new ArrayList<>();

    while(!queue.isEmpty()){
        List<String> currentPath = queue.poll();

        // If we move to next level, remove
        previously used words
        if(currentPath.size() > currentLevel){
            currentLevel = currentPath.size();
            for(String word : usedAtCurrentLevel){
                set.remove(word);
            }
            usedAtCurrentLevel.clear();
        }
```

TC -

SC -

Page No. :

Date :

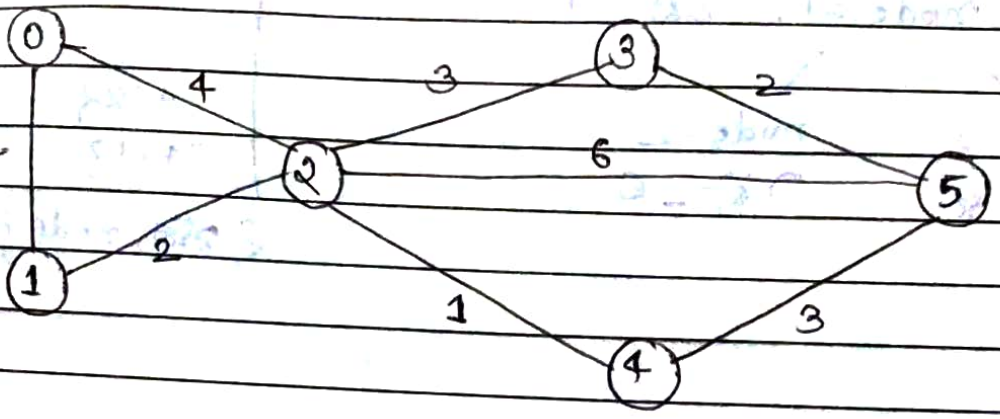
```
String lastWord = currentPath.get(currentPath.size()-1);

//if endWord is reached
if (lastWord.equals(endWord)){
    if (result.isEmpty()){
        result.add(new ArrayList<>(currentPath));
    }
    else if ( result.get(0).size() ==
              currentPath.size()){
        result.add(new ArrayList<>(currentPath));
    }
}

char wordArray[] = lastWord.toCharArray();

for(int i = 0; i<wordArray.length; i++){
    char originalChar = wordArray[i];

    for(char ch = 'a'; ch<='z'; ch++){
        wordArray[i] = ch;
        String nextWord = new String(wordArray);

        if(set.contains(nextWord)){
            currentPath.add(nextWord);
            queue.offer(new ArrayList<>(currentPath));
            usedAtCurrentLevel.add(nextWord);
            currentPath.remove(currentPath.size()-1);
        }
    }
    wordArray[i] = originalChar;
}
return result;
```

DIJKSTRA IS NOT APPLICABLE IN NEGATIVE GRAPH WEIGHT CYCLE

Page No. :

Date :

25. Dijkstra's Algorithm –



src = 0

Adj List

0 → {1,4} {2,4}
1 → {0,4} {2,2}
2 → {0,4} {1,2} {3,3} {4,1} {5,6}
3 → {2,3} {5,2}
4 → {2,1} {5,3}
5 → {2,6} {3,2} {4,3}

— USING PQ —

Step – 01

dist = | 0 | ∞ | ∞ | ∞ | ∞ | ∞ |
0 1 2 3 4 5

4 4

node = 0, dist = 0

node = 1, dist = 4

node = 2, dist = 4

| {4,2} |
| {4,1} |
| ~~{0,0}~~ |

min-Heap {dist, node}

Page No. :
Date :

Step-02 :-

node = 1, dist = 4

node = 0, dist = 8

node = 2, dist = 6

{4, 2}
~~{4, 1}~~

{dist, node}

Step-03 :-

node = 2, dist = 4

node = 5, dist = ∞

node = 0, dist = 8

node = 1, dist = 6

node = 3, dist = 7

node = 4, dist = 5

{10, 5}
{5, 4}
{7, 3}
~~{4, 2}~~

{dist, node}

dist =

0	1	2	3	4	5
0	4	4	~~∞~~ 7	~~0~~ 5	~~∞~~ 10

Step 04 :- node = 4, dist = 5

node = 2, dist = 6

node = 5, dist = 8

{8, 5}
~~{5, 4}~~
{7, 3}
{10, 5}

{dist, node}

PQ

dist =

0	1	2	3	4	5
0	4	4	7	5	~~10~~ 8

Step-05 :- node = 3, dist = 7

- node = 2, dist = 10
- node = 5, dist = 9

~~{7,3}~~
{8,5}
{10,5}
PQ

Step-06 :- node = 5, dist = 8

- node = 2, dist = 14
- node = 3, dist = 10
- node = 4, dist = 11

— NOT TO BE CONSIDERED —

~~{8,5}~~
{10,5}
PQ

Step-07 :- node = 5, dist = 10

- node = 2
- node = 3
- node = 4

— NOT TO BE CONSIDERED —

Page No. :
Date :

// Code :-

```
public int[] dijkstra(int V, int edges[][], int S){
    // Build adjacency list
    ArrayList<int[]> adj[] = new ArrayList[V];
    for(int i=0; i<V; i++){
        adj[i] = new ArrayList<>();
    }

    for(int edge[] : edges){
        int u = edge[0];
        int v = edge[1];
        int wt = edge[2];

        adj[u].add(new int[]{v, wt});
    }

    PriorityQueue<int[]> pq = new PriorityQueue<>
                (a, b) -> a[0] - b[0]);
    int dist[] = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[S] = 0;
    pq.offer(new int[]{0, S});

    while(!pq.isEmpty()){
        int curr[] = pq.poll();
        int d = curr[0];
        int node = curr[1];

        if(d > dist[node]){
            continue;
        }
```

```
for (int e[] : adj[node]) {
    int next = e[0];
    int w = e[1];

    if (d + w < dist[next]) {
        dist[next] = d + w;
        pq.offer(new int[] {dist[next], next});
    }
}
}
return dist;
}
```

TC- O(E log V)

SC- O(V+E)

↓

no of edges
adj list

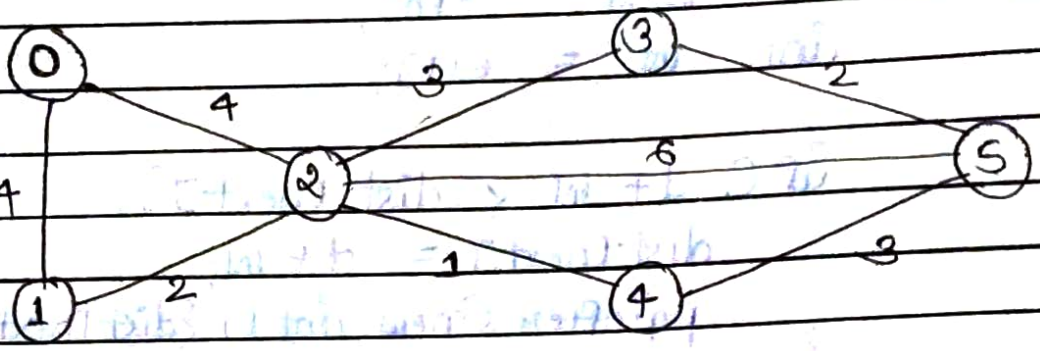
In Java, using TreeSet or HashSet it is not possible. //

Page No. :
Date :

Dijkstra's Algorithm –

– Using Set –

src = 0



dist = | 0 | ∞ | ∞ | ∞ | ∞ | ∞ |
0 1 2 3 4 5

{4,1}
{4,2}
{0,0}

Set DS
{dist, node}

Step-01 node=0, dist=0

node=1 dist=4

node=2 dist=4

dist = | 0 | 4 | 4 | ∞ | ∞ | ∞ |
0 1 2 3 4 5

Step-02 node=1 dist=4

node=0 dist=8

node=2 dist=6

{4,1}
{4,2}

[Erase → Already existing parts]

Page No. :
Date :

Step-03 — node=2 diff=4.

node=3 diff=7

node = 4 diff= 5

node = 5 diff=10

{5, 4}
{7, 3}
{10, 5}

diff = 2

0	4	4	~~7~~	~~5~~	~~10~~
0	1	2	3	4	5

Step-04 — node = 4, diff = 5

node = 2 diff = 6

node = 5 diff = ~~8~~

~~{5, 4}~~
{7, 3}
{10, 5}

diff =

0	4	4	~~7~~	5	~~10~~ 8
0	1	2	3	4	5

Step-05

Remove {10, 5} from the set {This is the advantage of Using Set}

Page No. :
Date :

~~25~~ — Time Complexity Analysis —

$O(V * C\log \text{Heapsize}) + ne * \log(\text{Heap size}))$

$\Rightarrow O((V * C\log \text{Heapsize}) * (ne + 1))$

$\Rightarrow O(V * V * \log(\text{Heapsize}))$

$\Rightarrow O(V^2 * \log(\text{Heapsize}))$

$\Rightarrow O(V^2 * \log(V^2))$

$\Rightarrow O(V^2 * 2\log V)$

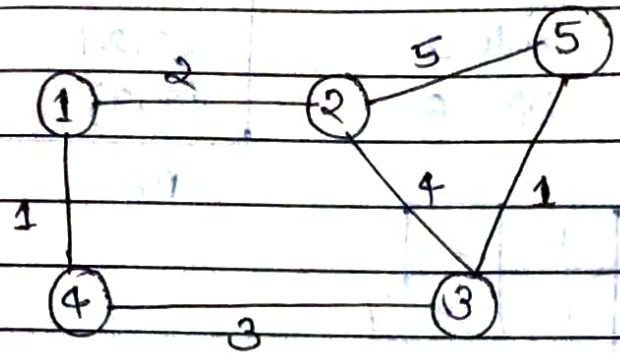
$\Rightarrow O(E * 2 * \log V) \approx O(E \log V)$

{ $E = V^2 =$ Total Edges }

Page No. :

Date :

26. Print Shortest Path - {Dijkstra Algo}



src = 1

dist = 5

Ans: $1 \rightarrow 4 \rightarrow 3 \rightarrow 5$

Step-01-

parent -

1	2	3	4	5

dist[] -

0	∞	∞	∞	∞
1	2	3	4	5

PQ {dist, node}

{1, 4}
{2, 2}
{0, 1}

node = 1 dist = 0

node = 2 dist = 2

node = 4 dist = 1

dist =

0	2	∞	1	∞
1	2	3	4	5

parent -

1	1	3	1	5

Step-02-

node = 4, dist = 1

node = 1 dist = 2

node = 3 dist = 4

PQ

{4, 3}
{1, 4}
{2, 2}

dist =

0	2	4	1	∞
1	2	3	4	5

parent -

1	1	4	1	5

Page No. :
Date :

Step-03 node = 2, dist = 2

node = 1	node = 5	node = 3
dist = 4	dist = 7	dist = 6

PQ: {7,5} {2,2} {4,3}

parent –	1	1	4	1	~~5~~ 2
dist –	0	2	4	1	~~8~~ 7

Step-04 node = 4, dist = 4

node = 4	node = 2	node = 5
dist = 7	dist = 8	dist = 5

PQ: {5,5} ~~{4,3}~~ {7,5}

parent –	1	1	4	1	~~2~~ 3
dist –	0	2	4	1	~~7~~ 5

Step-05 node = 5, dist = 5

node = 2	node = 3
dist = 10	dist = 6

PQ: ~~{5,5}~~ ~~{7,5}~~

Step-06 — node = 5, dist = 7

will give nothing

Empty PQ

Now start from the destination which is 5

5 is coming from 3
3 is coming from 4
4 is coming from 1
1 is coming from 0.

(1) → (4) → (3) → (5)

// Code :-

Page No. :
Date :

```
public List<Integer> shortestPath(int n, int m,
                    int edges[][]){
List<List<int[]>> adj = new Arraylist<>();
for(int i = 0; i<=n; i++){
    adj.add(new Arraylist<>());
}

for(int i=0; i<m; i++){
    int u = edges[i][0];
    int v = edges[i][1];
    int wt = edges[i][2];

    adj.get(u).add(new int[]{v, wt});
    adj.get(v).add(new int[]{u, wt});
}

// Min heap : {distance, node}
PriorityQueue<int[]> pq =
    new PriorityQueue<>((a,b) -> a[0]-b[0]);

int dist[] = new int[n+1];
int parent[] = new int[n+1];

Arrays.fill(dist, (int) 1e9);

for(int i = 1; i<=n; i++){
    parent[i] = i;
}

dist[1] = 0;
pq.add(new int[]{0,1});
```

Page No. :
Date :

TC - O(E log V) + O(N)
SC - O(V+E)

```
while (!pq.isEmpty()){
    int cur[] = pq.poll();
    int dis = cur[0];
    int node = cur[1];

    for (int it[] : adj.get(node)){
        int adjNode = it[0];
        int edgeWeight = it[1];

        if (dis + edgeWeight < dist[adjNode]){
            dist[adjNode] = dis + edgeWeight;
            pq.add(new int[]{dist[adjNode], adjNode});
            parent[adjNode] = node;
        }
    }
}

List<Integer> path = new ArrayList<>();
if (dist[n] == (int) 1e9){
    path.add(-1);
    return path;
}
int node = n;
while (parent[node] != node){
    path.add(node);
    node = parent[node];
}
path.add(1);
Collections.reverse(path);
return path;
}
```

Page No. :

Date :

27. Shortest Distance in a Binary maze -

	0	1	2	3
0	1	1	1	1
1	1	1	0	1
2	1	1	1	1
3	1	1	0	0
4	1	0	0	0

src = {0, 1}

dest = {2, 2}

Ans - 3

— Initial Configuration —

0	~~1~~	0	~~1~~	∞
1	∞	~~1~~	∞	∞
2	∞	∞	∞	∞
3	∞	∞	∞	∞
4	∞	∞	∞	∞

dist 0 0

{1, 0, 0}
{1, 1, 1}
{1, 0, 2}
{0, ~~0~~, 1}

{dist, {row, col}}

Queue

~~0 0~~

(0, 1) dist = 0

(0, 2) dist = 1

(1, 1) dist = 1

(0, 0) dist = 1

Using Queue :- Because Queue already store everything in a increasing Order

Page No. :

Date :

① (0,2) dist = 1

(0,3) dist = 2

(0,1) dist = 1

(2,0,3)
(1,0,0)
(1,1,1)
~~(1,0,2)~~

Queue

② (1,1) dist = 1

(0,1) dist = 2

(2,1) dist = 2

(2,1,0)
(2,2,1)
(2,0,3)
(1,0,0)
~~(1,1,1)~~

Queue

	0	1	2	3
0	1	0	1	2
1	2	1	∞	∞
2	∞	2	∞	∞
3	∞	∞	∞	∞
4	∞	∞	∞	∞

dist ∩ ∪

③ (0,0) dist = 1

(0,1) dist = 2

(1,0) dist = 2

{3,1,3}
(2,1,0)
(2,2,1)
~~(2,0,3)~~
~~(1,0,0)~~

Queue

Page No. :
Date :

(4) (0,3) dist = 2

(1,3) (0,2)

dist = 3 dist = 3

0	1	0	1	2
1	2	1	∞	3
2	∞	2	∞	∞
3	∞	∞	∞	∞
4	∞	∞	∞	∞

dist 0 1

(5) (2,1) dist = 2

(1,1) (2,2)

dist = 3 dist = 3

C
(3,1,3)
(2,1,0)
(2,2,1)

It is Equal to my Destination which is {2,2}

{Return distance = 3}

Page No. :
Date :

// Code -

```
public int shortestPath(int grid[][], int source[],
                        int destination[]){

    int n = grid.length;
    int m = grid[0].length;

    if( source[0] == destination[0] &&
        source[1] == destination[1] ){
        return 0;
    }

    int dist[][] = new int[n][m];
    for(int i = 0; i<n; i++){
        Arrays.fill(dist[i], Integer.MAX_VALUE);
    }

    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{source[0], source[1]});
    dist[source[0]][source[1]] = 0;

    int dr[] = {-1, 0, 1, 0};
    int dc[] = {0, 1, 0, -1};

    while(!q.isEmpty()){
        int cur[] = q.poll();
        int r = cur[0];
        int c = cur[1];
```

```
for (int i=0; i<4; i++){
    int nr = r + drow[i];
    int nc = c + dcol[i];

    if (nr >= 0 && nr < n &&
        nc >= 0 && nc < m &&
        grid[nr][nc] == 1 &&
        dist[r][c] + 1 < dist[nr][nc]){
        dist[nr][nc] = dist[r][c] + 1;

        if( nr == destination[0] &&
            nc == destination[1]){
            return dist[nr][nc];
        }
        q.offer(new int[]{nr, nc});
    }
}
}
return -1;
}
```

TC- $O(4 N*M)$ ~~+ O(N*M)~~

SC- $O(N*M) + O(N*M)$

dist Queue

Page No. :
Date :

28. Path With Minimum Effort –

Ex 01

src → (1) 2 2
3 8 2
5 3 (5) ← dest

1 2 2 2 5
1 0 0 3 = 3 effort

1 3 8 2 5
2 5 6 3 = 6 effort

1 3 5 3 5
2 2 2 2 → 2 effort

Ans ⇒ 2

Ex 02

1	2	1	1	1
1	2	1	2	1
1	2	1	2	1
1	1	1	2	1

Ans ⇒ 0

— Dry Run —

Initial Configuration –

1	2	2
3	8	2
5	3	5

0	∞	∞
∞	∞	∞
∞	∞	∞

diff[0][0]

PQ (Min Heap)
(0,0,0)
{diff, row, col}

Page No. :
Date :

① (0,0) diff = 0

(0,1) diff = 1

(1,0) diff = 2

(2,1,0)
(1,0,1)
(0,0,0)

0	1	∞
2	∞	∞
∞	∞	∞

dist(1,1)

② (0,1) diff = 1

(0,2) diff = 0

(1,1) diff = 6

(6,1,1)
(1,0,2)
(1,0,1)
(2,1,0)

0	1	1
2	6	∞
∞	∞	∞

③ (0,2) diff = 1

(1,2) diff = 0

(1,1) diff = 0

(1,1,2)
(1,0,2)
(2,1,0)
(6,1,1)

0	1	1
2	6	1
∞	∞	∞

dist(1,1)

Page No. :
Date :

④ (1,2) diff = 1

(0,2) diff = 0

(2,2) diff = 3

0	1	1
2	6	1
∞	∞	3

{3,2,2}
~~(1,1,2)~~
(2,1,0)
(6,1,1)

⑤ (1,0) diff = 2

(0,0) diff = 2

(1,1) diff = 5

(2,0) diff = 2

0	1	1
2	~~6~~5	1
2	∞	3

(2,2,0)
{5,1,1}
~~(2,1,0)~~
(3,2,2)
(6,1,1)

⑥ (2,0) diff = 2

(2,1) diff = 2

X

0	1	1
2	5	1
2	2	3

(2,2,1)
~~(2,2,0)~~
(3,2,2)
(5,1,1)
(6,1,1)

(7) (2,1) diff = 2

(1,1) diff = 5

(2,2) diff = 2

0	1	1
2	5	1
2	2	~~2~~ 2

(2,2,2)
~~(2,2,1)~~
(3,2,2)
(5,1,1)
(6,1,1)

(8) (2,2) diff = 2

Page No. :
Date :

// Code :-

```
public int MinimumEffort(int heights[][]){

    PriorityQueue<int[]> pq = new PriorityQueue<>
            (Comparator.comparingInt(a -> a[0]));

    int n = heights.length;
    int m = heights[0].length;

    int dist[][] = new int[n][m];
    for(int row[] : dist){
        Arrays.fill(row, Integer.MAX_VALUE);
    }

    dist[0][0] = 0;
    pq.add(new int[]{0,0,0});

    int dx[] = {-1,0,1,0};
    int dc[] = {0,1,0,-1};

    while(!pq.isEmpty()){
        int current[] = pq.poll();
        int diff = current[0];
        int row = current[1];
        int col = current[2];

        if(row == n-1 && col == m-1){
            return diff;
        }
```

```
for (int i=0; i<4; i++) {
    int newr = row + dr[i];
    int newc = col + dc[i];

    if (newr >= 0 && newc >=0 &&
        newr < n && newc < m ) {

        int newEffort = Math.max(Math.abs
            (heights[row][col] - heights[newr][newc]),
                diff);

        if (newEffort < dist[newr][newc]) {
            dist[newr][newc] = newEffort;
            pq.add(new int[] {newEffort,
                newr, newc});
        }
    }
}
return 0;
}
```

insertion, deletion in PQ

TC- O(4 * N * m * log(N * M))

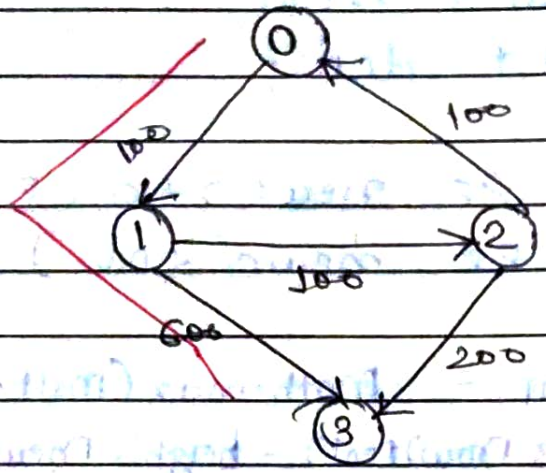
SC- O(N * M)

↓

for mat + pq

Page No. :
Date :

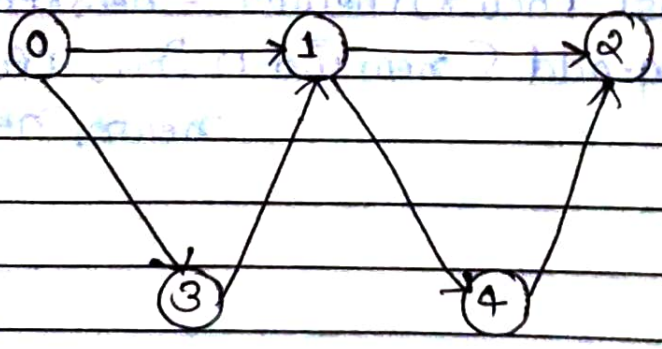
29. Cheapest Flights Within K Steps -



src = 0
dest = 3
K = 1

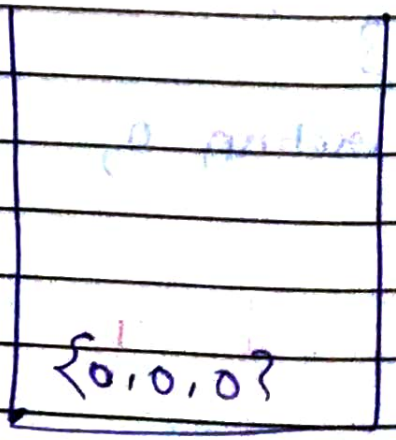
Ans — 700

— Intution —



dist[] =

0	∞	∞	∞	∞
0	1	2	3	4

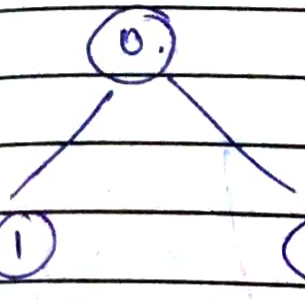


{stops, node, dist}

Page No. :

Date :

(1)



dist = 0 stop = 0

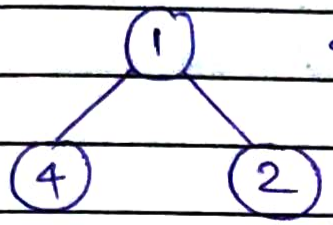
dist = 5 stop = 1

dist = 2 stop = 1

dist	0	5	∞	2	∞
	0	1	2	3	4

{1,3,2}
{1,1,5}
~~{0,6,10}~~

(2)



dist = 1 stop = 1

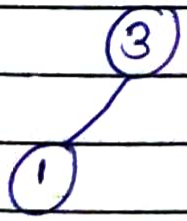
dist = 6 stop = 2

dist = 10 stop = 2

dist =	0	5	10	2	6
	0	1	2	3	4

{2,2,10}
~~{2,4,6}~~
~~{1,3,2}~~
~~{1,1,5}~~

(3)

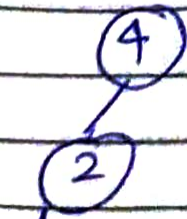


dist = 2 stop = 1

dist = 5

stops = 2

(4)



dist = 6 stops = 2

dist = 7

stops = 3

dist =	0	5	~~10~~ 7	2	6
	0	1	2	3	4

{3,2,7}
{2,2,10}
~~{2,4,6}~~

Page No. :
Date :

(5) (2) Stops = 2

I have reached this is my destination. No need to go beyond.

{3,2,7}
~~{2,12,10}~~

{Stops, node, dis}

(6) {3, 2, 7}

I have already reached.

// Code -

Page No. :
Date :

```
public int CheapestFlight(int n, int flights[][],
        int src, int dst, int K) {

List<List<int[]>> adj = new ArrayList<>();
for(int i=0; i<n; i++){
    adj.add(new ArrayList<>());
}

for(int flight[]: flights){
    adj.get(flight[0]).add(new int[]{flight[1],
                                     flight[2]});
}

Queue<int[]> q = new LinkedList<>();
q.offer(new int[]{0, src, 0});

int dist[] = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;

while(!q.isEmpty()){
    int current[] = q.poll();
    int stops = current[0];
    int node = current[1];
    int cost = current[2];

    if(stops > K){
        continue;
    }
```

TC - O(N)
SC - O(E+V)

```
for (int adjNode[] : adj.get(node)) {
    int nextNode = adjNode[0];
    int edWt = adjNode[1];

    if (cost + edWt < dist[nextNode]
            && stops <= K) {
        dist[nextNode] = cost + edWt;
        q.offer(new int[] {stops + 1,
            nextNode, cost + edWt});
    }
}
}
if (dist[dst] == Integer.MAX_VALUE) {
    return -1;
}
return dist[dst];
}
```

Page No. :
Date :

30. Minimum Multiplication to Reach End -

① start = 3 end = 30 arr[] = {2, 5, 7}

Explanation :- start → 3 $\xrightarrow[①]{\times 2}$ 6 $\xrightarrow[②]{\times 5}$ 30

Ans = 2

② start = 7 end = 66175 arr[] = {3, 4, 65}

Explanation :- start 7 $\xrightarrow[①]{\times 3}$ 21 $\xrightarrow[②]{\times 3}$ 63 $\xrightarrow[③]{\times 65}$ 4095 $\xrightarrow[④]{\times 65}$ 266175

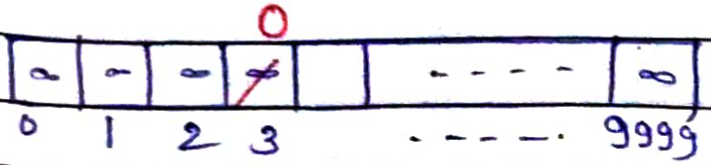
266175 % 10^5 ⇒ 66175

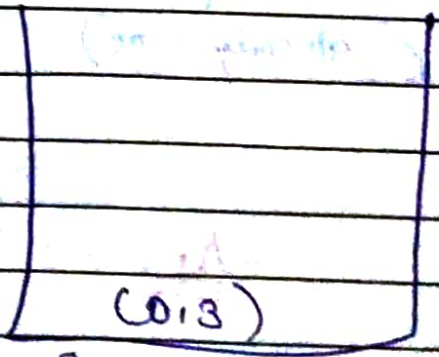
Ans - 4

— Intution —

start = 3 end = 75 mod = 100000

arr = [2, 5, 7]





dist[]

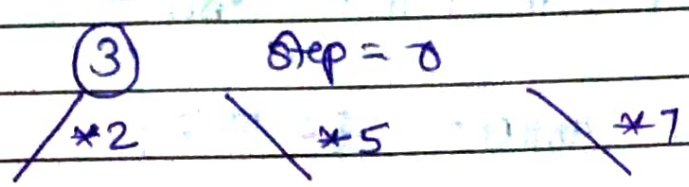
{steps, num}

~~min heap~~

Queue

Page No. :
Date :

(1)

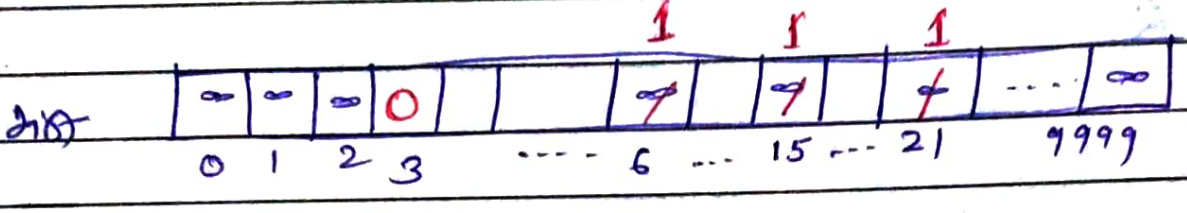


6, Step=1 (15), Step=1 (21), Step=1

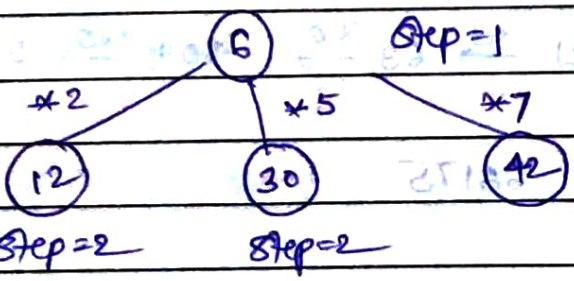
(1,21)
(1,15)
(1,6)
(0,3)

{steps, num}

min heap



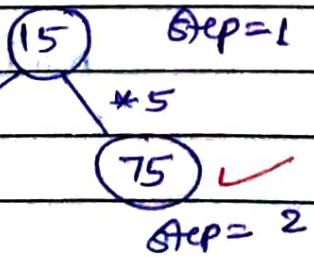
(2)



(1,21)
(1,15)
(1,6)

steps, num

Queue



(already done)

Ans ⇒ 2

//Code —

Page No. :
Date :

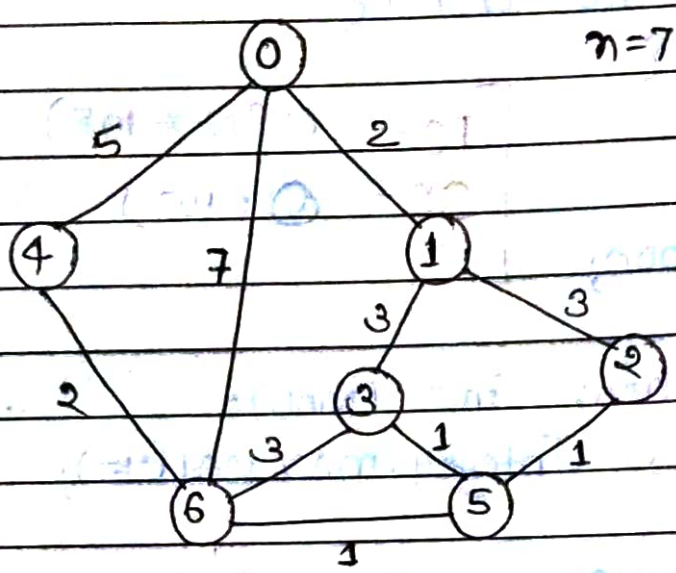
```
public int minimumMultiplications(int arr[], int start,
                                  int end) {
    if (start == end) {
        return 0;
    }
    int MOD = 100000;

    int dist[] = new int[MOD];
    Arrays.fill(dist, Integer.MAX_VALUE);

    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start);
    dist[start] = 0;

    while (!queue.isEmpty()) {
        int curr = queue.poll();

        for (int mul : arr) {
            int next = (int) ((long) curr * mul % MOD);

            if (dist[curr] + 1 < dist[next]) {
                dist[next] = dist[curr] + 1;

                if (next == end) {
                    return dist[next];
                }

                queue.offer(next);
            }
        }
    }

    return -1;
}
```

TC- $O(N * 10^5)$

SC- $O(10^5)$

Page No. :
Date :

31. Number of Ways to Arrive at Destination –



n=7 src=0 dest=6

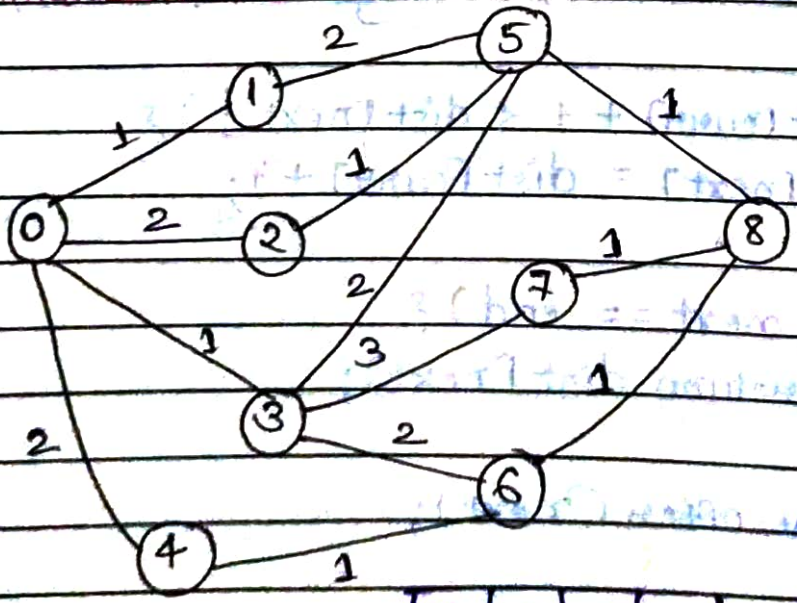
Ans ⇒ 4

Explanation :–

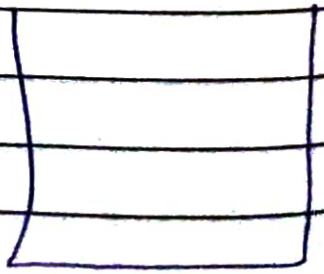
0 → 6
0 → 4 → 6
0 → 1 → 2 → 5 → 6
0 → 1 → 3 → 5 → 6

– Intuition –

Initialization :–



src = 0



{dist, node}
min-heap

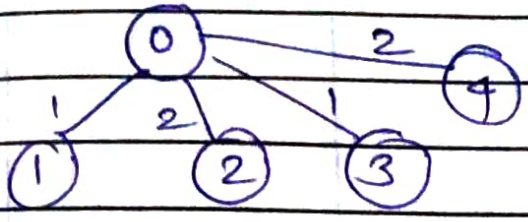
ways =
dest =

1	0	0	0	0	0	0	0	0
0	∞	∞	∞	∞	∞	∞	∞	∞
0	1	2	3	4	5	6	7	8

Page No. :

Date :

(1)

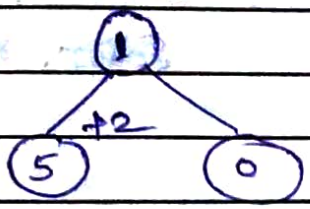


ways =	1	1	1	1	1	0	0	0	0
मार्ग =	0	1	2	1	2	∞	∞	∞	∞
	0	1	2	3	4	5	6	7	8

{2,4}
{1,3}
{2,2}
{1,1}
{0,0}

minheap

(2)



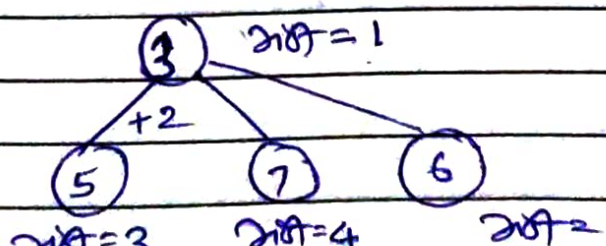
मार्ग = 3 मार्ग = 2

ways =	1	1	1	1	1	1	0	0	0
मार्ग =	0	1	2	1	2	3	∞	∞	∞
	0	1	2	3	4	5	6	7	8

{3,6}
{4,7}
{2,4}
{2,2}
{1,3}
{1,1}

Min-heap

(3)



ways =	1	1	1	1	1	2	1	1	0
मार्ग =	0	1	2	1	2	3	3	4	∞
	0	1	2	3	4	5	6	7	8